

Scholaris

Multi-tenant School ERP — Project Documentation

Generated May 9, 2026

41 migrations · 12 test suites · 135 tests

Platform Control Layer Phases 1–18 + Maturity push

Scholaris — Project Documentation

Purpose. *This document is the single source of truth for what the Scholaris platform is, what's been built, how the pieces fit together, and what's deferred. It's written for the engineer who picks the project up next — whether that's you in three months or a new collaborator tomorrow. It describes **what exists**, not what we hope to build.*

*Sections that begin with **⚠ Deferred** describe known gaps with explicit reasoning. Don't treat them as bugs — they're conscious tradeoffs documented for the next planning cycle.*

Table of contents

1. [Project overview](#)
 2. [Tech stack](#)
 3. [Architectural principles](#)
 4. [Repository layout](#)
 5. [Database schema](#)
 6. [Authentication & security model](#)
 7. [The two surfaces — School Dashboard vs Platform Control Layer](#)
 8. [Platform Control Layer \(Phases 1–10\)](#)
 9. [Maturity push \(Phases 11–18\)](#)
 10. [Backend module reference](#)
 11. [Frontend route + component reference](#)
 12. [API reference](#)
 13. [Notification system](#)
 14. [Background job system](#)
 15. [Feature flags](#)
 16. [Email templates](#)
 17. [Audit log](#)
 18. [Migrations chronology](#)
 19. [Test coverage](#)
 20. [Environment variables](#)
 21. [Operational guide](#)
 22. [Known limitations & deferred work](#)
 23. [Future scope / roadmap](#)
 24. [Decision log — key architectural choices](#)
-

1. Project overview

Scholaris is a multi-tenant school ERP delivered as SaaS. One backend

- database serves many schools (tenants); each school's data is isolated by `schoolId` on every domain row.

The system has two distinct surfaces:

- **School Dashboard** (`/dashboard/...`) — what admins, staff, and teachers at a school use day-to-day. Tenant-scoped. Soft visual language (school primary color, rounded cards, glass effects). Built iteratively over Phases 1–14 of school-side product work (see commit history pre-platform layer).
- **Platform Control Layer** (`/platform/...`) — what the SaaS operator (`SUPER_ADMIN` role) uses to manage tenants. Cross-tenant. Slate / operational visual language. Built across Phases 1–10 (Platform Roadmap) + Phases 11–18 (Maturity push).

Geographic context. Built for Nepal — supports Bikram Sambat (BS) calendar via `bikram-sambat-js`, NPR currency formatting (`₹.`), and Indian-style number grouping (lakh / crore). The codebase is locale-aware where it matters (receipts, marksheets, dates) and English elsewhere.

Status. All ten Platform Control Layer phases are shipped end-to-end. Maturity push has shipped Phases 11–18 with 9 of 10 email templates, session management, ops dashboard, and full test coverage of the security-critical surfaces.

2. Tech stack

Backend

Layer	Choice	Notes
Runtime	Node.js (any 20+ LTS)	Tested on Node 20
Framework	NestJS 11	Modular DI, controllers, guards, pipes
Language	TypeScript 5 (strict)	<code>tsc --noEmit</code> is a CI gate
ORM	Prisma 6	PostgreSQL provider
Database	PostgreSQL	Multi-tenant by <code>schoolId</code>
Auth	JWT (<code>@nestjs/jwt</code> + <code>passport-jwt</code>)	7-day default TTL
Password hashing	bcrypt (12 rounds)	<code>HashingService</code> abstraction
Validation	<code>class-validator</code> + <code>class-transformer</code>	Global <code>ValidationPipe</code>
Scheduling	<code>@nestjs/schedule</code>	Cron triggers for jobs
Rate limiting	<code>@nestjs/throttler</code>	Custom user-aware guard
Email (dev)	Console provider	Logs to stdout

Layer	Choice	Notes
Email (prod)	<code>nodemailer</code> (SMTP)	Lazy-loaded
Testing	Jest	12 suites, 135 tests

Frontend

Layer	Choice	Notes
Framework	Next.js 14 (App Router)	Two route groups: <code>(dashboard)</code> + <code>platform</code>
Language	TypeScript 5	Strict mode
Styling	Tailwind CSS	Custom design tokens
Icons	<code>lucide-react</code>	Used throughout
Toasts	<code>sonner</code>	Single shared toaster
Charts	Custom SVG (<code>Sparkline</code>) + light wrappers	No heavy chart lib
State	Local React state + localStorage	No Redux / Zustand
Date utility	<code>bikram-sambat-js</code>	BS calendar conversion

Infrastructure (current dev setup)

- Local PostgreSQL on `localhost:5432`
- Backend on port `3001` (dev)
- Frontend on port `3000` (dev)
- Migrations applied via `npx prisma migrate deploy`
- No CI/CD pipeline configured yet

3. Architectural principles

These are the rules I've enforced consistently. Violating them is a code-review red flag.

3.1. Tenant isolation is mandatory

Every domain row carries `schoolId`. Every read query filters by it. Every write writes it. The **only** code that bypasses this is the Platform Control Layer (`/platform/*`), which is `SUPER_ADMIN` -gated.

3.2. Append-only audit trails

Two tables are append-only by design:

- `school_subscriptions` — every plan change / renewal / extension is a NEW row. The "current" subscription is the most-recent by `createdAt` . We never edit history.
- `platform_audit_events` — every operator-tier write produces a row. Includes actor, target, before/after JSON snapshots, reason, IP, user-agent. Never edited.

3.3. Defense-in-depth on auth

Every authenticated request passes through this guard chain:

```

UserAwareThrottlerGuard      (per-user-id rate limit)
  ↓
JwtAuthGuard                 (token signature + watermark + session)
  ↓
RolesGuard                   (when @Roles() declared)
  ↓
FeatureFlagsGuard            (when @RequireFeature() declared)
  ↓
MaintenanceModeGuard         (writes only; blocks during maintenance)
  ↓
Controller method

```

3.4. Best-effort side effects

Side effects (audit emits, notification enqueues, email sends) **never roll back** the underlying action. A delivery failure logs but doesn't fail the transaction. Audit failures swallow errors and log to stderr. This is operator-grade resilience: school data integrity > observability.

3.5. Idempotency at write time

Producers that may fire twice (network retries, double-clicks) get unique constraints to collapse on:

- `payments.(schoolId, clientRequestId)` — payment idempotency key
- `notifications.(templateKey, dedupeKey)` — notification dedupe
- `jobs.(name, dedupeKey)` — job dedupe
- `sessions` — single row per token (id is the dedupe key implicitly)

3.6. Two surfaces, deliberately separate

The school dashboard and the platform control layer never share components, layouts, or styling tokens. Cross-pollination is a bug. Reasoning: different audiences, different security postures, different visual languages.

School-side is `frontend/components/ui/` ; platform-side is `frontend/components/platform-ui/` .

3.7. Comments explain WHY, not WHAT

Every file has a header block explaining its purpose, key tradeoffs, and pointers to related code. Inline comments mark non-obvious decisions. Don't add line-by-line comments — code that needs them should be refactored.

4. Repository layout

```

school management system/
├── backend/
│   ├── prisma/
│   │   ├── schema.prisma          # Single source of truth for DB
│   │   ├── migrations/           # 41 migrations, chronologically named
│   │   ├── seed-super-admin.ts    # Seeds the platform owner
│   │   └── ...
│   ├── src/
│   │   ├── app.module.ts          # Root module – every Module imported here
│   │   ├── main.ts                # Bootstrap (CORS, validation, trust proxy)
│   │   ├── auth/                  # Login, JWT, registration
│   │   ├── platform/              # Platform Control Layer
│   │   ├── feature-flags/         # @Global feature gating
│   │   ├── notifications/         # @Global notification system
│   │   ├── sessions/              # @Global session tracking
│   │   ├── health/                # @Global operator pulse
│   │   ├── common/
│   │   │   ├── filters/           # AllExceptionsFilter
│   │   │   ├── jobs/              # @Global background queue
│   │   │   ├── maintenance/       # MaintenanceModeGuard
│   │   │   ├── throttle/          # UserAwareThrottlerGuard
│   │   │   ├── decorators/        # @CurrentUser, @Roles
│   │   │   ├── hashing/           # bcrypt wrapper
│   │   │   └── money/              # amountInWords
│   │   ├── academic-session/      # School year management
│   │   ├── announcement/          # School-wide notices (feature-gated)
│   │   ├── attendance/            # Daily attendance
│   │   ├── class/                 # Class management
│   │   ├── dashboard/             # School-side aggregated stats
│   │   ├── exams/                 # NEB-style exam + results
│   │   ├── fees/                  # Fee structures, assignments, payments
│   │   ├── grading/               # NEB letter-grade calc
│   │   ├── promotion/             # End-of-year promotion (feature-gated)
│   │   ├── school/                # Tenant config (logo, contact, etc.)
│   │   ├── section/               # Class subdivisions
│   │   ├── student/               # Student profiles
│   │   ├── subject/               # School subject catalog
│   │   ├── teacher/               # Teacher profiles
│   │   ├── teaching-assignment/   # Teacher → class/section/subject
│   │   └── user/                  # Per-school user mgmt (admin → invite teachers etc)
│   ├── test/                      # E2E tests (none yet)
│   ├── package.json
│   └── tsconfig.json
├── frontend/
│   ├── app/
│   │   ├── layout.tsx             # Root layout
│   │   ├── page.tsx               # Landing
│   │   ├── login/                 # /login
│   │   ├── (dashboard)/           # School-side route group
│   │   │   ├── layout.tsx         # Dashboard chrome (Sidebar + Topbar)
│   │   │   ├── dashboard/         # /dashboard (role-aware)
│   │   │   ├── students/          # /students CRUD
│   │   │   ├── teachers/
│   │   │   ├── classes/
│   │   │   ├── subjects/
│   │   │   └── attendance/

```

└─ exams/	# /exams/marks, /exams/create
└─ fees/	# Receipt generation
└─ analytics/	# Admin analytics
└─ announcements/	
└─ settings/	
└─ page.tsx	# Main settings (admin-only)
└─ sessions/	# ACADEMIC sessions (year mgmt)
└─ devices/	# User sessions / devices (Phase 17)
└─ offline/	# Offline queue diagnostics
└─ platform/	# Platform Control Layer
└─ layout.tsx	# Platform chrome (slate)
└─ page.tsx	# /platform overview
└─ schools/	
└─ page.tsx	# Schools list
└─ [id]/page.tsx	# School detail page (Phase 12)
└─ subscriptions/	# Cross-tenant subscription view
└─ audit/	# Audit log
└─ features/	# Feature flag matrix
└─ health/	# System health dashboard
└─ notifications/	# Notification Center (Phase 14)
└─ operations/	# Ops dashboard (Phase 18)
└─ marketing/	# Public landing pages (skeletal)
└─ marksheet/	# Public marksheet view
└─ receipts/	# Public receipt view
└─ results/	# Public result view
└─ components/	
└─ ui/	# School-side primitives
└─ platform-ui/	# Platform-side primitives (Phase 5 maturity)
└─ platform/	# Platform-specific (dialogs, etc.)
└─ layout/	# Sidebar, Topbar, NotificationsBell
└─ analytics/	# Filter UI, charts wrappers
└─ academic-session/	# Session selector
└─ calendar/	# BS / AD date pickers
└─ charts/	# Sparkline, gauge, etc.
└─ dashboard/	# AdminDashboardView, TeacherDashboardView
└─ documents/	# Marksheet renderer, etc.
└─ exams/	# Marks entry grid
└─ fees/	# Receipt renderer
└─ impersonation/	# ImpersonateUserDialog, banner
└─ maintenance/	# MaintenanceBanner (Phase 17)
└─ students/	# Student CRUD form
└─ sync/	# Offline sync UI
└─ teachers/	# Teacher CRUD form
└─ theme/	# ThemeToggle (light/dark)
└─ classes/	# Class form
└─ lib/	
└─ api.ts	# Typed HTTP client
└─ auth.ts	# Login state, logout, impersonation
└─ platform.ts	# Platform API client + types
└─ sessions.ts	# /me/sessions client (Phase 17)
└─ features.tsx	# FeaturesProvider + useFeatures hook
└─ currency.ts	# NPR formatter
└─ date.ts	# AD ↔ BS conversion helpers
└─ grading.ts	# NEB grade lookup
└─ offline-queue.ts	# IndexedDB queue
└─ sync-engine.ts	# Background sync runner
└─ ... per-domain clients (students.ts, teachers.ts, etc.)	
└─ package.json	
└─ tsconfig.json	
└─ docs/	

```

└─ platform/
   └─ PROJECT.md           # ← this file
   └─ architecture.md     # Module map + request lifecycle
   └─ notifications.md    # Notification flow
   └─ jobs.md             # Background queue
   └─ impersonation.md    # Impersonation safety model

```

5. Database schema

Schema lives in `backend/prisma/schema.prisma`. Every model is documented with `///` doc comments inline; this section gives the high-level map.

5.1. Core tenant + auth

Model	Purpose	Key fields
School	Tenant. Every domain row FK's to this.	<code>id</code> , <code>name</code> , <code>slug</code> (unique), <code>logoUrl</code> , <code>address</code> , <code>phone</code> , <code>email</code> , <code>status</code> (enum), <code>expiresAt</code> , <code>maintenanceMode</code> (Phase 17), <code>featureOverrides</code> (JSON, Phase 5)
User	Authentication record.	<code>id</code> , <code>email</code> (unique), <code>password</code> (bcrypt hash), <code>role</code> (enum), <code>schoolId</code> , <code>tokensValidAfter</code> (Phase 9 watermark)
Session	Per-token tracking (Phase 17 follow-up).	<code>id</code> , <code>userId</code> , <code>lastActiveAt</code> , <code>ip</code> , <code>userAgent</code> , <code>revokedAt</code> , <code>revokedReason</code>

5.2. Domain profiles

Model	Purpose	Key fields
Student	Student record.	<code>firstName</code> , <code>lastName</code> , <code>symbolNumber</code> (unique per school), <code>gender</code> , <code>dateOfBirth</code> , <code>parentName</code> , <code>contactNumber</code> , <code>address</code> , <code>admissionDate</code> , <code>userId</code> (optional 1:1 with User), <code>classId</code> , <code>sectionId</code>
Teacher	Teacher record.	<code>name</code> , <code>userId</code> (REQUIRED 1:1), <code>assignments[]</code> (TeachingAssignment[])

5.3. Academic hierarchy

Model	Purpose
Class	Class within a school (e.g. "Class 8")
Section	Subdivision of a class (e.g. "8-A")
Subject	School-owned subject catalog (e.g. "Mathematics")

Model	Purpose
TeachingAssignment	One row per (teacher, class, optional section, optional subject). Drives all teacher scoping.
AcademicSession	School year. Exactly one <code>isActive</code> per school. <code>isLocked</code> is precondition for promotion.
StudentAcademicRecord	Per-student per-session immutable history. Set at promotion.

5.4. Attendance + exams

Model	Purpose
Attendance	One row per (student, date). <code>status</code> is <code>PRESENT</code> / <code>ABSENT</code> .
Exam	Named exam (e.g. "Term 1"). FK to School + AcademicSession.
ExamSubject	Per-exam subject row with <code>theoryFullMarks</code> + <code>practicalFullMarks</code> .
Result	One row per (student, subject). Stores <code>theoryMarks</code> , <code>practicalMarks</code> , derived <code>percentage</code> + <code>letterGrade</code> + <code>gradePoint</code> . <code>absent</code> flag for NEB "did not appear" (NG).

5.5. Fees

Model	Purpose
FeeStructure	School-owned fee catalog (e.g. "Term 1 Tuition"). Optional class scope.
FeeAssignment	One row per (student, fee). Snapshots <code>amount</code> at assignment time. Optional <code>discountType</code> + <code>discountValue</code> .
Payment	Cashier-recorded payment. Auto-generated <code>receiptNumber</code> (RCPT-YYYY-NNNN). Idempotent via <code>clientRequestId</code> . Refunds are SEPARATE rows with negative amount + <code>refundOfPaymentId</code> link.

5.6. Communications

Model	Purpose
Announcement	School-wide notice (feature-gated).

5.7. Platform control layer

Model	Purpose
SchoolSubscription	Append-only subscription history. Most recent row = current plan.
PlatformAuditEvent	Append-only operator action log.

5.8. Maturity infrastructure

Model	Purpose
Notification	One per logical event. Idempotent via (templateKey, dedupeKey) .
NotificationDelivery	One per channel attempt. Tracks status (QUEUED → SENT / FAILED / SKIPPED).
Job	Persistent background queue. (name, dedupeKey) unique.

5.9. Enums

```

enum Role {
    SUPER_ADMIN ADMIN STAFF TEACHER STUDENT PARENT
}

enum SchoolStatus {
    ACTIVE TRIAL SUSPENDED EXPIRED
}

enum SubscriptionPlan {
    TRIAL MONTHLY YEARLY UNLIMITED
}

enum BillingCycle {
    MONTHLY YEARLY ONE_TIME PERPETUAL
}

enum PlatformAuditAction {
    SCHOOL_STATUS_CHANGED           // Phase 8
    IMPERSONATION_STARTED           // Phase 7
    IMPERSONATION_ENDED             // Phase 7
    SUBSCRIPTION_CREATED            // Phase 4
    FEATURE_FLAG_CHANGED            // Phase 5
    USER_FORCE_LOGOUT              // Phase 9
    SCHOOL_FORCE_LOGOUT             // Phase 9
    ADMIN_PASSWORD_RESET           // Phase 9
    SCHOOL_MAINTENANCE_TOGGLED     // Phase 17
}

enum NotificationChannel {
    EMAIL SMS IN_APP WHATSAPP
}

enum NotificationDeliveryStatus {
    QUEUED SENDING SENT FAILED SKIPPED
}

enum NotificationSeverity {
    INFO SUCCESS WARNING ERROR CRITICAL
}

enum JobStatus {
    PENDING SCHEDULED RUNNING SUCCEEDED FAILED DEAD
}

// Domain enums
enum AttendanceStatus { PRESENT ABSENT }
enum Gender { MALE FEMALE OTHER }
enum LetterGrade { A_PLUS A B_PLUS B C_PLUS C D NG }
enum FeeFrequency { MONTHLY ONE_TIME }
enum StudentSessionStatus { PROMOTED RETAINED LEFT }
enum PaymentMethod { CASH BANK ESEWA OTHER }
enum DiscountType { PERCENT FIXED }
enum PaymentStatus { ACTIVE REFUNDED VOID }

```

6. Authentication & security model

6.1. Token lifecycle

```
User submits email + password
↓
AuthService.login()
├─ Validates password (bcrypt.compare)
├─ On failure → records to HealthService failed-login buffer
├─ Checks tenant gate (PlatformService.assertSchoolCanLogin)
│   └─ SUSPENDED / EXPIRED schools reject (except SUPER_ADMIN)
├─ For TEACHER: requires at least one TeachingAssignment
├─ Creates Session row (Phase 17) – captures IP + userAgent
├─ Issues JWT with payload:
│   { userId, role, schoolId, sid: <session-id>, iat, exp }
└─
```

6.2. Per-request validation chain

```
Bearer token arrives
↓
JwtStrategy.validate()
├─ Looks up user by payload.userId
├─ Verifies user.schoolId matches payload.schoolId (defense vs token forgery)
├─ Watermark check:
│   └─ if user.tokensValidAfter exists AND payload.iat < watermark → 401
├─ Session check (if payload.sid present):
│   └─ if Session.findActive(sid) returns null → 401
│       └─ else throttle-touch lastActiveAt (1min throttle)
└─ Returns AuthenticatedUser (id, email, role, schoolId, sessionId, impersonatedBy?)
```

6.3. Three layers of token control

Mechanism	Use case	Granularity	When
JWT <code>exp</code>	Natural expiry	Per-token	Default 7d
<code>users.tokensValidAfter</code>	"Kill ALL my user's tokens"	Per-user	Phase 9 force-logout, password reset
<code>sessions.revokedAt</code>	"Kill THAT one token"	Per-session	Phase 17 user/admin revoke

All three are checked on every request. Failing any → 401.

6.4. Roles

Role	Scope	Notes
<code>SUPER_ADMIN</code>	Cross-tenant	Platform owner. NOT tied to any school's lifecycle. Bypasses every feature flag + maintenance mode. Cannot be impersonated.
<code>ADMIN</code>	Single school	Full access within tenant. Manages users, fees, students, classes, etc.

Role	Scope	Notes
STAFF	Single school	Mid-tier academic role. Can manage subjects, exams, marks for ANY class. Cannot manage users, fees, students, classes.
TEACHER	Single school + assignments	Acts ONLY on classes/sections in their TeachingAssignment rows.
STUDENT / PARENT	Reserved	Read-only roles, not yet wired into the UI.

6.5. Impersonation (Phase 7)

`SUPER_ADMIN` can sign in as a school user. The mechanism + safety rules are documented in detail at `docs/platform/impersonation.md`. Key points:

- New JWT carries the TARGET's identity + sentinels (`impersonatedBy` , `impersonationStartedAt`)
- Cannot impersonate another `SUPER_ADMIN`
- Cannot impersonate self
- Cannot nest (no impersonation-of-impersonator)
- Cannot enter SUSPENDED / EXPIRED tenants (must reactivate first)
- Domain audit columns (`createdById` on payments etc.) get the TARGET user — intentional, since impersonation is for "reproduce what the admin sees"
- Platform audit captures every start + end with full actor/target/timestamps
- Banner is rendered ABOVE the topbar on every dashboard page

6.6. Rate limiting

- **Default:** 600 req/min per **user** (custom `UserAwareThrottlerGuard` keys by `req.user.id` when authenticated, falls back to `req.ip`)
- `/auth/login` : 10/min/IP (named `auth` bucket)
- `/auth/register` : 5/hour/IP (named `register` bucket)
- `/platform/health` : `@SkipThrottle()` — operator polling

Rationale documented in `backend/src/common/throttle/user-aware-throttler.guard.ts`.

6.7. Maintenance mode (Phase 17)

Per-tenant boolean (`schools.maintenanceMode`) enforced by global `MaintenanceModeGuard`. When ON:

- Read methods (`GET` , `HEAD` , `OPTIONS`) pass.
- Write methods (`POST` , `PATCH` , `PUT` , `DELETE`) reject with 503 + message "This school is in maintenance mode. Reads are allowed; writes are paused."
- `SUPER_ADMIN` always bypasses (operator needs writes to FIX maintenance).
- `/platform/*` paths always bypass (operator console).

- School-side dashboard renders an amber banner ABOVE the topbar.
 - Toggled from the school detail page; audited via `SCHOOL_MAINTENANCE_TOGGLED`.
-

7. The two surfaces

7.1. School Dashboard

URL prefix. `/(dashboard)/...` (Next.js route group)

Audience. ADMIN / STAFF / TEACHER users at one school.

Visual language. Soft, school-themed:

- School primary color from CSS variables
- Rounded cards with shadows (`Card` from `components/ui/`)
- Glass effects in the topbar (`glass` utility class)
- Larger empty states with illustration
- Friendly, welcoming copy

Layout.

- Fixed topbar (calendar toggle, session selector, notifications bell, user menu)
- Collapsible sidebar (responsive: drawer on mobile)
- Optional banners: ImpersonationBanner, MaintenanceBanner

Pages (selected).

- `/dashboard` — role-aware (admin sees school-wide; teacher sees own classes)
- `/students` — CRUD with filtering, search, bulk operations
- `/teachers` , `/classes` , `/sections` , `/subjects` — admin CRUDs
- `/attendance` — daily attendance grid per (class, date)
- `/exams/marks` — bulk marks entry grid
- `/exams/create` , `/results/ledger` — exam lifecycle
- `/fees` — fee structures, assignments, payments + receipt printing
- `/announcements` — feature-gated
- `/analytics` — admin-only, 5-tab dashboard with cross-module insights
- `/settings` — admin-only mega-page (school config, users, subjects, sessions, etc.)
- `/settings/devices` — user-level (Phase 17)

7.2. Platform Control Layer

URL prefix. `/platform/...`

Audience. SUPER_ADMIN only.

Visual language. Operational, dense:

- Slate base palette, no school accent
- Square / sharp cards (`SectionCard` from `components/platform-ui/`)
- No gradients, minimal animation
- Information-dense, fast-scanning
- Direct, technical copy

Layout.

- Sticky topbar with "Platform" identity + Sign out + "School view" exit
- Sidebar with status pills next to ready/coming-soon items

Pages (every one is shipped).

Route	Purpose	Phase
<code>/platform</code>	Cross-tenant overview	6
<code>/platform/schools</code>	All-tenant list with status filter + row actions	3
<code>/platform/schools/[id]</code>	Per-tenant operational command center	12
<code>/platform/subscriptions</code>	Cross-tenant subscription view (expiring / expired / no plan / active)	4
<code>/platform/audit</code>	Filterable audit log with side-drawer detail	8
<code>/platform/features</code>	Cross-tenant feature flag matrix	5
<code>/platform/health</code>	Live operator pulse (uptime, DB, errors, login failures)	10
<code>/platform/notifications</code>	Notification Center inbox	14
<code>/platform/operations</code>	Ops cockpit (revenue, growth, queue, risk)	18

8. Platform Control Layer (Phases 1–10)

Built sequentially. Each phase ships a coherent slice end-to-end.

Phase 1 — SUPER_ADMIN role

- New `SUPER_ADMIN` enum value on `Role`
- Seed script `prisma/seed-super-admin.ts` to create the first owner
- Migration: `20260515000000_platform_layer`

Phase 2 — Platform shell

- Separate `/platform` Next.js layout with slate visual language

- Three-state access gate (no token / token but not SUPER_ADMIN / allowed)
- Sidebar with "ready" / "soon" status pills

Phase 3 — School management

- `GET /platform/schools` — paginated list with search + status filter
- `GET /platform/schools/:id` — single school detail
- `PATCH /platform/schools/:id/status` — flip lifecycle (with audit + reason)
- Backend enforces "reason required" for SUSPENDED / EXPIRED
- Frontend dialog mirrors the requirement

Phase 4 — Subscriptions

- `school_subscriptions` table (append-only)
- `SubscriptionService.create()` writes new period + flips `school.status` per rules:
 - TRIAL plan → status TRIAL
 - Future-ending non-TRIAL → status ACTIVE (unless SUSPENDED — SUSPENDED is operator-tier)
 - Past-ending → status EXPIRED
- `school.expiresAt` denormalized cache of current `endDate`
- `ManageSubscriptionDialog.tsx` UI
- Cross-tenant `/platform/subscriptions` page bucketed (Expiring soon / Expired / No plan / Active)
- Migration: `20260518000000_subscriptions`

Phase 5 — Feature flags

- `Schools.featureOverrides` (JSON) + per-tenant overrides
- Three-layer resolution (override > subscription > catalog default) in `FeatureFlagsService`
- `@RequireFeature(FeatureKey.X)` decorator + `FeatureFlagsGuard`
- Catalog: analytics, announcements, promotion (default ON); sms, transport, hostel (default OFF, marked Soon)
- `/me/features` school-side endpoint for cached flag map
- `/platform/features` cross-tenant matrix UI with three-state cells
- `<FeatureGate>` page-level guard component
- Sidebar entries hide when feature disabled
- SUPER_ADMIN bypasses every flag
- Migration: `20260519000000_feature_flags`

Phase 6 — Platform analytics (basic)

- `GET /platform/overview` — cross-tenant KPIs + 12-month school growth trend
- `/platform` overview page composes status counts + aggregate usage + sparkline

Phase 7 — Impersonation

- `ImpersonationService` mints token with target's identity + sentinels
- `POST /platform/impersonate/:userId` (start)
- `POST /platform/impersonate/end` (returns fresh SUPER_ADMIN token)
- Impersonation banner on every dashboard page
- Cannot impersonate SUPER_ADMIN, self, or into SUSPENDED/EXPIRED tenant
- Cannot nest impersonation
- Migration: `20260517000000_impersonation_audit_actions`

Phase 8 — Audit logs

- `platform_audit_events` table (append-only)
- `PlatformAuditService.record()` — single ingestion path; swallows errors
- `GET /platform/audit` — paginated, filterable (action, actor, target, date, free-text search)
- Frontend audit page with custom per-action ChangeSummary + side-drawer for full row detail
- Migration: `20260516000000_platform_audit_events`

Phase 9 — Security controls

- `users.tokensValidAfter` watermark column
- `JwtStrategy` checks watermark on every request
- `SecurityService.forceLogoutUser()` / `forceLogoutSchool()` / `resetPassword()`
- `POST /platform/users/:id/force-logout`
- `POST /platform/schools/:id/force-logout` (reason required, 3+ chars)
- `POST /platform/users/:id/reset-password` (returns plaintext temp password ONCE)
- Refuses SUPER_ADMIN targets across all three actions
- Bulk school force-logout skips SUPER_ADMIN rows
- `SecurityDialog.tsx` UI with school-name confirmation gate for bulk action
- `ResetPasswordResultModal` with click-to-copy + acknowledge gate
- Three new audit actions
- Rate limiting: `@nestjs/throttler` with named buckets for `/auth/login` + `/auth/register`
- `main.ts` adds `app.set('trust proxy', 1)` for real client IP
- Migration: `20260520000000_security_controls`

Phase 10 — System health

- `HealthService` (in-memory, @Global): uptime, memory, DB probe, error ring buffer (200 cap), failed-login ring buffer (200 cap)
- Status rollup: green / yellow / red
- `AllExceptionsFilter` switched to `APP_FILTER` (DI-injected) so it can record errors to HealthService

- `AuthService.login` records failed logins with IP + reason (`invalid_credentials` / `school_blocked`)
 - `GET /platform/health` (skipped from throttle)
 - `/platform/health` page with 30s polling + three-tier status banner + scrollable error/login feeds
 - Compact health summary card on `/platform` overview
 - Top source-IP rollup for failed logins (last 60min)
-

9. Maturity push (Phases 11–18)

Once the 10-phase platform was complete, a maturity push hardened operational gaps. Numbering continues for traceability.

Phase 11 — Test hardening

- Test infrastructure was minimal (1 test). Now: 12 suites, 135 tests.
- Coverage focus: security-critical surfaces (auth gates, impersonation rules, force-logout watermarks, feature flag resolution)
- Real Prisma mocks (in-memory map-backed) over schema-shaped fixtures

Phase 12 — Platform school detail page

- `GET /platform/schools/:id/snapshot` — bundled per-school analytics + unified activity feed (payments + audit + subscriptions, max 40 items)
- `/platform/schools/[id]` page with 7 sections + sticky action bar:
 1. Overview (logo monogram, identity, contact, dates)
 2. Subscription & billing (KvBlocks + history + days-remaining banner)
 3. Feature flags (per-row toggle, optimistic updates, three-state cycle)
 4. Platform analytics (KPI row + 30-day fee/attendance sparklines)
 5. System health (computed warnings list, severity-tinted card)
 6. Recent activity (unified feed with kind-aware icons)
 7. Administration (sticky action bar with status-aware buttons)

Phase 13 — Email expansion (9 templates shipped)

- Template registry pattern in `backend/src/notifications/templates/`
- Branded `wrapEmail()` layout (table-based, inline CSS, mobile-safe)
- Templates: `password_reset` , `school_created` , `subscription_expiring` , `school_suspended` , `payment_receipt` , `subscription_renewed` , `school_reactivated` , `plan_changed` , `security_alert`
- HTML + plain-text + optional in-app rendering per template

- Wired to: `AuthService.registerAdmin` , `SecurityService.resetPassword` , `PlatformService.updateSchoolStatus` (suspend + reactivate), `SubscriptionService.create` (renewed vs plan_changed), `FeesService.recordPayment` (receipt)

Phase 14 — Notification Center

- `notifications.severity` , `notifications.title` , `notifications.readAt` columns added (Phase 14 migration)
- `NotificationCenterService` + `NotificationCenterController`
- Routes: `GET /platform/notifications` , `GET .../unread-count` , `GET .../:id` , `PATCH .../:id/read` , `PATCH .../:id/unread`
- `/platform/notifications` page with severity multi-select, unread-only filter, day-grouped list, side-drawer with per-channel deliveries
- Sidebar entry with Bell icon

Phase 15 — Background job system

- `Job` model + `JobStatus` enum (Phase 15 migration)
- `JobsModule` (@Global): `JobQueueService` , `JobRegistry` , `JobRunnerService`
- DB-backed queue with `FOR UPDATE SKIP LOCKED` claim semantics
- Exponential backoff retry (30s, 2m, 8m, 32m, ... capped 1h, $\pm 20\%$ jitter)
- Idempotency via `(name, dedupeKey)` unique constraint
- `JobNonRetryableError` for fast-fail
- `OnApplicationShutdown` settles in-flight job (30s timeout)
- `JOBS_AUTOSTART=false` env disables poll for tests
- Two real handlers shipped:
 - `notification.send_delivery` (per-delivery dispatch with retry)
 - `platform.subscription_expiring_notice` (per-school per-threshold)
- Cron refactored to enqueue jobs (`SubscriptionExpiringJob`)

Phase 16 — Platform analytics expansion

- `PlatformAnalyticsService` with 4 buckets:
 - **Revenue** — MRR / ARR using code-level `PLAN_MONTHLY_PRICE_NPR` map (TRIAL=0, MONTHLY=5,000, YEARLY=4,000, UNLIMITED=0)
 - **Growth** — schools/month for 12 months, 30-day delta vs prior 30-day, feature adoption ratios
 - **System** — queue depth (`JobQueueService.stats()`), recent failed jobs (24h), notification volume by severity
 - **Risk** — suspended / expired / expiring soon / inactive (30d+ no activity proxy via `users.updatedAt`)
- `GET /platform/analytics` endpoint

-  Deferred: churn rate (needs subscription state-change history), successful-login trends (needs `lastSeenAt` column)

Phase 17 — Security hardening

- **Maintenance mode:** `schools.maintenanceMode` field + `MaintenanceModeGuard` (global `APP_GUARD`, soft read-only), `/platform/schools/:id/maintenance` toggle, school-side `MaintenanceBanner`, `SCHOOL_MAINTENANCE_TOGGLED` audit action
- **Sessions** (Phase 17 follow-up):
 - `Session` model + `sessions` table (Phase 17 follow-up migration)
 - `SessionService` (CRUD + throttled `touch` + bulk revoke)
 - JWT payload extended with `sid` claim
 - `JwtStrategy.validate` looks up session, throttle-touches `lastActiveAt`
 - `POST /auth/logout` — revokes calling session
 - `GET /me/sessions` + `/me/sessions/:id/revoke` + `/me/sessions/revoke-others`
 - `GET /platform/users/:id/sessions` + revoke endpoint (operator-tier)
 - `/settings/devices` school-side UI with "Sign out everywhere else"
 - "Active devices" link added to topbar user menu (all roles)
 - User-agent parser (browser + OS) for session list display

Phase 18 — Ops Dashboard

- `/platform/operations` page composing `PlatformAnalyticsPayload` + `HealthPayload`
- Widgets: status banner, KPI row (MRR / new schools / at-risk / queue depth), plan distribution, notifications volume, risk panel, queue panel, recent failed jobs, feature adoption
- 60s polling
- Sidebar entry with Cog icon

Phase 19 — Documentation

- `docs/platform/architecture.md` — module map + request lifecycle
- `docs/platform/notifications.md` — flow + adding templates + center API
- `docs/platform/jobs.md` — queue concepts + handler patterns
- `docs/platform/impersonation.md` — security model + audit attribution
- `docs/PROJECT.md` — this document

Phase 5 (Maturity) — Design system

- `frontend/components/platform-ui/` — primitives for the platform layer:
 - `PageHeader` (title + breadcrumbs + actions slot)
 - `SectionCard` (bordered card with title bar, 4 tones)
 - `StatsGrid` + `StatCard` (KPI tile)
 - `StatusPill` + `SchoolStatusPill` + `PlanPill` (severity-tinted chips)
 - `PanelEmptyState` , `PanelErrorState` , `PanelLoadingState`
 - `SkeletonLine` , `SkeletonRows`
 - `FilterToolbar` (search + filters strip with active chips)
- All 6 platform pages migrated to use these primitives
- Net: ~400 lines of inline markup eliminated, guaranteed visual consistency

10. Backend module reference

Every Nest module under `backend/src/` . Listed in dependency order (roughly).

10.1. Foundation modules

Module	Path	Notes
<code>DatabaseModule</code>	<code>database/</code>	Provides <code>PrismaService</code> (global)
<code>ConfigModule</code>	(built-in)	Loaded in <code>app.module</code> with <code>configuration.ts</code>
<code>HashingModule</code>	<code>common/hashing/</code>	<code>HashingService</code> wraps bcrypt
<code>TeacherScopeModule</code>	<code>common/auth/</code>	<code>TeacherScopeService</code> for assignment-based access checks

10.2. Auth + identity

Module	Path	Exports
<code>AuthModule</code>	<code>auth/</code>	<code>AuthService</code> , <code>JwtStrategy</code> , JWT module, Passport module
<code>UserModule</code>	<code>user/</code>	<code>UserService</code> for school-side user CRUD

10.3. Domain modules

Module	Path	Notes
<code>SchoolModule</code>	<code>school/</code>	Tenant config (logo upload, contact, etc.)
<code>StudentModule</code>	<code>student/</code>	Student CRUD with class/section assignment

Module	Path	Notes
TeacherModule	teacher/	Teacher CRUD; required <code>userId</code> link
ClassModule	class/	Class management
SectionModule	section/	Section management
SubjectModule	subject/	School-owned subject catalog
TeachingAssignmentModule	teaching-assignment/	Teacher → class/section/subject
AcademicSessionModule	academic-session/	School year mgmt + lock + activate
AttendanceModule	attendance/	Daily attendance recording
GradingModule	grading/	NEB letter-grade lookup (no DB writes; pure logic)
ExamsModule	exams/	Exam + ExamSubject + Result CRUD
FeesModule	fees/	FeeStructure + FeeAssignment + Payment + refund
DashboardModule	dashboard/	School-side aggregated stats
AnnouncementModule	announcement/	Feature-gated school notices
PromotionModule	promotion/	Feature-gated end-of-year flow

10.4. Cross-cutting infrastructure (all `@Global()`)

Module	Path	Purpose
PlatformModule	platform/	Platform Control Layer (NOT <code>@Global</code> ; <code>SUPER_ADMIN</code> -gated routes)
FeatureFlagsModule	feature-flags/	<code>FeatureFlagsService</code> , <code>FeatureFlagsGuard</code> , <code>/me/features</code>
HealthModule	health/	<code>HealthService</code> for operator pulse
NotificationsModule	notifications/	<code>NotificationService</code> , <code>NotificationCenterService</code> , channels, providers, templates, controller
SessionsModule	sessions/	<code>SessionService</code> + <code>SessionsController</code>
JobsModule	common/jobs/	<code>JobQueueService</code> , <code>JobRegistry</code> , <code>JobRunnerService</code>

10.5. Global guards / filters / interceptors

Mechanism	Class	Registered as
Exception handler	<code>AllExceptionsFilter</code>	<code>APP_FILTER</code> in <code>AppModule.providers</code>
Rate limit	<code>UserAwareThrottlerGuard</code>	<code>APP_GUARD</code> in <code>AppModule.providers</code>

Mechanism	Class	Registered as
Maintenance mode	<code>MaintenanceModeGuard</code>	<code>APP_GUARD</code> in <code>AppModule.providers</code>

11. Frontend route + component reference

11.1. Route groups

Next.js App Router with two route groups:

- `app/(dashboard)/` — school-side. Wraps every page with `DashboardLayout` (Sidebar + Topbar + ImpersonationBanner + MaintenanceBanner + FeaturesProvider).
- `app/platform/` — platform-side. Wraps with `PlatformLayout` (slate Topbar + Sidebar with ready/soon pills).

Plus public/standalone routes:

- `app/login/` — login page (no layout)
- `app/marketing/` — public marketing pages (skeletal)
- `app/marksheet/` , `app/receipts/` , `app/results/` — public document viewers

11.2. Auth state (client-side)

Stored in `localStorage` :

Key	Shape	Set by
<code>scholaris:token</code>	string	<code>auth.login()</code> , <code>beginImpersonation()</code>
<code>scholaris:user</code>	<code>SafeUser</code> JSON	same
<code>scholaris:school</code>	<code>SchoolSummary</code> JSON	same
<code>scholaris:impersonation</code>	<code>ImpersonationContext</code> JSON	<code>beginImpersonation()</code> only
<code>scholaris:features</code>	<code>Record<string, boolean></code>	<code>FeaturesProvider</code> cache

Cross-tab safety: `DashboardLayout` listens for `storage` events on `scholaris:token` / `scholaris:user` . Any change → hard navigation to `/login` so both tabs converge.

11.3. Frontend lib

File	Purpose
<code>lib/api.ts</code>	Typed HTTP client. Attaches JWT, handles 401 (clears + redirects), 403 (logs but doesn't redirect).

File	Purpose
<code>lib/auth.ts</code>	Login/logout (now async — calls <code>/auth/logout</code>), impersonation helpers, localStorage I/O
<code>lib/platform.ts</code>	All platform-tier API client + types
<code>lib/sessions.ts</code>	<code>/me/sessions</code> client (Phase 17)
<code>lib/features.tsx</code>	<code>FeaturesProvider</code> + <code>useFeatures()</code> hook + cached read
<code>lib/currency.ts</code>	NPR formatter (<code>₹. 12,345.50</code> with Indian grouping)
<code>lib/date.ts</code>	AD ↔ BS conversion helpers via <code>bikram-sambat-js</code>
<code>lib/grading.ts</code>	NEB letter-grade lookup (mirror of backend logic)
<code>lib/amount-in-words.ts</code>	"Rupees Twelve Thousand..." — receipts
<code>lib/csv.ts</code>	CSV export helpers
<code>lib/offline-queue.ts</code> + <code>lib/sync-engine.ts</code>	IndexedDB-backed offline write queue + drainer
<code>lib/use-dashboard-data.ts</code>	School-side dashboard data hook
<code>lib/{students,teachers,classes,...}.ts</code>	Per-domain API clients

11.4. Component primitives

School-side (`components/ui/`) — soft, school-themed:

- `Card` , `CardHeader` , `CardTitle` , `CardDescription` , `CardContent` , `CardFooter`
- `Button` (multiple variants: default, ghost, outline, destructive; sizes: sm, md, lg)
- `Input`
- `Modal`
- `Skeleton`
- `Table`
- `EmptyState` (school-themed gradient illustration)
- `Toaster` (`sonner` wrapper)
- `OnboardingChecklist`

Platform-side (`components/platform-ui/`) — slate, operational:

- `PageHeader` (title + breadcrumbs + actions slot, regular/compact)
- `SectionCard` (bordered card with title bar; tones: default/warning/danger/success)
- `StatsGrid` + `StatCard` (KPI tile with icon/delta/loading/href)
- `StatusPill` + `SchoolStatusPill` + `PlanPill`
- `PanelEmptyState` , `PanelErrorState` , `PanelLoadingState`

- `SkeletonLine` , `SkeletonRows`
- `FilterToolbar` (search + filters + active-chip strip)
- Re-exports through `components/platform-ui/index.ts`

Cross-surface components (`components/`):

- `layout/Sidebar.tsx` — school-side sidebar with role-gated + feature-gated entries
- `layout/Topbar.tsx` — school-side topbar with user menu + Devices link
- `layout/NotificationsBell.tsx` — placeholder (school-side inbox future)
- `impersonation/ImpersonateUserDialog.tsx` — picker
- `impersonation/ImpersonationBanner.tsx` — sticky banner during impersonation
- `maintenance/MaintenanceBanner.tsx` — school-side maintenance notice
- `platform/ManageSubscriptionDialog.tsx` — subscription create + history
- `platform/SecurityDialog.tsx` — force-logout + reset password
- `platform/FeatureGate.tsx` — page-level feature flag wrapper
- `theme/ThemeToggle.tsx` — light/dark toggle
- `calendar/DualDate.tsx` , `calendar/CalendarToggle.tsx` — BS/AD date controls
- `academic-session/SessionSelector.tsx` , `AcademicSessionProvider.tsx`
- `dashboard/AdminDashboardView.tsx` , `dashboard/TeacherDashboardView.tsx`
- `analytics/DateRangeMenu.tsx` and tab components
- `documents/MarksheetRenderer.tsx` , `documents/ReceiptRenderer.tsx`
- `sync/SyncStatusBadge.tsx`
- `charts/Sparkline.tsx`
- Per-domain forms: `students/StudentForm.tsx` , `teachers/TeacherForm.tsx` , `classes/ClassForm.tsx` , etc.

12. API reference

Every endpoint. Grouped by surface.

12.1. Auth (`/auth`)

Method	Path	Auth	Notes
POST	<code>/auth/register</code>	none	Register new tenant + first ADMIN. Throttled 5/hour/IP.
POST	<code>/auth/login</code>	none	Login. Throttled 10/min/IP. Records failed logins to health.
POST	<code>/auth/logout</code>	JWT	Phase 17 — revokes calling session row.

12.2. School-side `/me/*`

Method	Path	Auth	Notes
GET	<code>/me/features</code>	JWT	Resolved feature map + catalog + tenant maintenance state
GET	<code>/me/sessions</code>	JWT	Phase 17 — list my sessions
POST	<code>/me/sessions/:id/revoke</code>	JWT	Phase 17
POST	<code>/me/sessions/revoke-others</code>	JWT	Phase 17 — log out everywhere except here

12.3. School-side domain (excerpt — full list in controllers)

Method	Path	Auth	Notes
GET	<code>/dashboard/summary</code>	JWT	School-wide aggregate (admin tier)
GET	<code>/dashboard/teacher-summary</code>	JWT, TEACHER	Teacher's own context
GET, POST, PATCH, DELETE	<code>/students*</code>	JWT	Standard CRUD
GET, POST, PATCH, DELETE	<code>/teachers*</code>	JWT, ADMIN	Includes <code>/teachers/me/assignments</code>
GET, POST, PATCH, DELETE	<code>/classes*</code> , <code>/sections*</code> , <code>/subjects*</code>	JWT, ADMIN	
GET, POST, DELETE	<code>/announcements*</code>	JWT (read), ADMIN (write); <code>@RequireFeature(Announcements)</code>	
GET, POST	<code>/attendance*</code>	JWT	
POST	<code>/promotion/run</code>	JWT, ADMIN; <code>@RequireFeature(Promotion)</code>	
GET, POST, PATCH	<code>/exams*</code> , <code>/results*</code>	JWT	

Method	Path	Auth	Notes
GET, POST, PATCH	/fees* , /payments*	JWT, ADMIN	
POST	/payments/:id/refund	JWT, ADMIN	
GET, POST, PATCH	/academic-sessions*	JWT, ADMIN	
GET, PATCH	/users*	JWT, ADMIN	
GET, PATCH, POST	/school*	JWT, ADMIN	Logo upload via /school/logo

12.4. Platform layer (@Roles(SUPER_ADMIN) on all)

Method	Path	Notes
GET	/platform/overview	Cross-tenant KPIs
GET	/platform/schools	Paginated list
GET	/platform/schools/:id	Detail row
GET	/platform/schools/:id/snapshot	Phase 12 — analytics + activity feed bundle
GET	/platform/schools/:id/users	Non-SUPER_ADMIN users at the school
GET	/platform/schools/:id/subscriptions	Plan history
POST	/platform/schools/:id/subscriptions	Create new period
PATCH	/platform/schools/:id/status	Change lifecycle (reason required for SUSPENDED/EXPIRED)
PATCH	/platform/schools/:id/maintenance	Phase 17 — toggle maintenance mode
GET	/platform/schools/:id/features	Resolved + layered features
PATCH	/platform/schools/:id/features	Set overrides
GET	/platform/features/catalog	Catalog only
GET	/platform/features	Cross-tenant matrix
GET	/platform/users/:id/sessions	Phase 17 — list any user's sessions
POST	/platform/users/:userId/sessions/:sessionId/revoke	Phase 17 — revoke one session

Method	Path	Notes
POST	<code>/platform/users/:id/force-logout</code>	Watermark hammer
POST	<code>/platform/schools/:id/force-logout</code>	Bulk school force-logout (reason required)
POST	<code>/platform/users/:id/reset-password</code>	Returns plaintext temp password ONCE
POST	<code>/platform/impersonate/:userId</code>	Start impersonation
POST	<code>/platform/impersonate/end</code>	End (returns fresh SUPER_ADMIN token)
GET	<code>/platform/audit</code>	Filterable + paginated
GET	<code>/platform/health</code>	Operator pulse (skipped from throttle)
GET	<code>/platform/analytics</code>	Phase 16 — revenue/growth/system/risk
GET	<code>/platform/notifications</code>	Phase 14 — list with filters
GET	<code>/platform/notifications/unread-count</code>	Badge counter
GET	<code>/platform/notifications/:id</code>	Detail + per-channel deliveries
PATCH	<code>/platform/notifications/:id/read</code>	Mark read
PATCH	<code>/platform/notifications/:id/unread</code>	Mark unread

13. Notification system

Full architecture documented at `docs/platform/notifications.md` . Summary here.

13.1. Producer-side API

```
await this.notifications.enqueue({
  templateKey: 'platform.password_reset',
  recipients: { email: 'foo@bar.com' },
  dedupeKey: `user:${userId}:reset:${eventTime}`, // optional
  severity: 'WARNING', // optional, defaults to INFO
  title: 'Password reset by support', // optional
  schoolId: '...', // optional, for filtering
  userId: '...', // optional, for filtering
  payload: { brand: this.config.get('mail.brand'), ... },
});
```

Returns `{ notification, deliveries, deduped: boolean }` . Errors during dispatch DO NOT propagate — they're recorded on the delivery row.

13.2. Channels

Channel	Status	Handler
EMAIL	Live	EmailChannel → EmailProvider (Console / SMTP)
IN_APP	Wired (no UI yet)	InAppChannel (no-op; rows are the inbox)
SMS	Reserved	(no handler) — SKIPPED
WHATSAPP	Reserved	(no handler) — SKIPPED

13.3. Email providers

EMAIL_PROVIDER injection token (in notifications/providers/email-provider.token.ts) bound by env via factory in NotificationsModule :

- MAIL_PROVIDER=console (default) → ConsoleEmailProvider (logs to stdout)
- MAIL_PROVIDER=smtp → SmtplibEmailProvider (lazy-loads nodemailer)

13.4. Template registry

In backend/src/notifications/templates/ :

- template-registry.ts — exports TEMPLATES map + getTemplate(key)
- layout.ts — wrapEmail() + emailHeading/Paragraph/Button/Mono/Divider
- 9 template files (see §16)

Adding a template: drop a <your>.template.ts , register in TEMPLATES , call notifications.enqueue({ templateKey: 'your.key', ... }) from your producer. Done.

13.5. Notification Center (Phase 14)

Operator-facing inbox at /platform/notifications . SUPER_ADMIN-gated. Backed by NotificationCenterService :

- List with severity multi-select + unread filter + schoolId filter + pagination
- Detail with payload + per-delivery state
- Mark read / unread
- Unread count for the bell badge

14. Background job system

Full architecture at docs/platform/jobs.md . Summary here.

14.1. Concepts

- **Producer** — calls `JobQueueService.enqueue({ name, payload, dedupeKey?, runAt? })`
- **Handler** — `@Injectable()` class implementing `JobHandler`, registered with `JobRegistry`
- **Runner** — `JobRunnerService` polls every 1s, claims with `FOR UPDATE SKIP LOCKED`
- **Queue** — `jobs` table

14.2. Lifecycle

```

PENDING (runAt <= now)
  ↓ JobRunner.claimNext (FOR UPDATE SKIP LOCKED)
RUNNING
  └─ handler resolved → SUCCEEDED
  └─ handler threw:
      └─ attempts < maxAttempts → PENDING with backoff
      └─ attempts >= maxAttempts OR JobNonRetryableError → FAILED
  (Operator-killed) → DEAD

```

14.3. Backoff

Exponential: 30s, 2m, 8m, 32m, ... capped at 1h, with $\pm 20\%$ jitter.

14.4. Test mode

`JOBS_AUTOSTART=false` disables the poll loop. Tests use `jobRunner.runOnceForTesting()` to drain deterministically.

14.5. Shipped handlers

Name	Owner	Purpose
<code>notification.send_delivery</code>	<code>notifications/handlers/</code>	Per-delivery dispatch with retry
<code>platform.subscription_expiring_notice</code>	<code>platform/jobs/</code>	Per-school per-threshold expiry email

14.6. Cron

`@nestjs/schedule` for triggers (not the job runner itself). One cron ships:

- `SubscriptionExpiringJob` — `@Cron(EVERY_DAY_AT_9AM)`. Scans schools in the ± 2 -week window of `expiresAt` and enqueues `platform.subscription_expiring_notice` jobs. The job runner drains them async.

15. Feature flags

Resolution stack (highest precedence first):

1. `schools.featureOverrides` — operator-set per-tenant override
2. `school_subscriptions.enabledFeatures` — current plan's flags
3. Catalog default — code-level `defaultEnabled` per feature

15.1. Catalog (in `backend/src/feature-flags/feature-catalog.ts`)

Key	Default	Coming Soon	Notes
<code>analytics</code>	ON	—	Admin analytics dashboard
<code>announcements</code>	ON	—	School-wide messaging
<code>promotion</code>	ON	—	End-of-year flow
<code>sms</code>	OFF	✓	Future feature
<code>transport</code>	OFF	✓	Future feature
<code>hostel</code>	OFF	✓	Future feature

15.2. Backend enforcement

```
@Controller('announcements')
@UseGuards(JwtAuthGuard, RolesGuard, FeatureFlagsGuard)
@RequireFeature(FeatureKey.Announcements)
export class AnnouncementController { ... }
```

`FeatureFlagsGuard` (in `feature-flags/`) resolves at request time via

`FeatureFlagsService.isEnabled(schoolId, key)` . `SUPER_ADMIN` bypasses every flag.

15.3. Frontend

- `FeaturesProvider` mounted in `(dashboard)/layout.tsx` fetches `/me/features` on mount
- `useFeatures()` / `useFeatureEnabled(key)` hooks read the cached map
- `<FeatureGate featureKey="..." featureLabel="...">` page-level wrapper
- Sidebar entries with `requiresFeature` field hide when off

15.4. Cross-tenant management

- `/platform/features` — three-state matrix (Inherit / Forced ON / Forced OFF)
- Per-school override view embedded in school detail page
- `FEATURE_FLAG_CHANGED` audit row on every override write (no-op writes skipped)

16. Email templates

In `backend/src/notifications/templates/` . All emit HTML + plain-text.

Key	Trigger	Recipient
<code>platform.school_created</code>	<code>AuthService.registerAdmin</code>	New school admin
<code>platform.password_reset</code>	<code>SecurityService.resetPassword</code>	Reset target
<code>platform.subscription_expiring</code>	<code>SubscriptionExpiringJob</code> cron (5 thresholds: 14, 7, 1, 0, -1 days)	School's first ADMIN
<code>platform.subscription_renewed</code>	<code>SubscriptionService.create</code> (when plan unchanged or first paid period)	School's first ADMIN
<code>platform.plan_changed</code>	<code>SubscriptionService.create</code> (when plan differs from previous)	School's first ADMIN
<code>platform.school_suspended</code>	<code>PlatformService.updateSchoolStatus</code> (→ SUSPENDED)	School's first ADMIN
<code>platform.school_reactivated</code>	<code>PlatformService.updateSchoolStatus</code> (→ ACTIVE from SUSPENDED/EXPIRED)	School's first ADMIN
<code>platform.security_alert</code>	(no producer wired yet — template ready for anomaly detection)	Variable
<code>school.payment_receipt</code>	<code>FeesService.recordPayment</code> (when student has linked user with email)	Student's user email

⚠ Deferred: `low_storage_warning` — needs storage tracking first. ⚠ Deferred: `refund_receipt` , `exam_published` , `invitation_email` — needs corresponding producers wired.

16.1. Email layout

`templates/layout.ts` exports:

- `wrapEmail({ brand, preheader, body })` — full HTML shell
- `emailHeading(text)` , `emailParagraph(text)` , `emailButton(label, href)`
- `emailMonoBlock(text)` , `emailDivider()`

Brand config (`MAIL_BRAND_*` env vars): `productName`, `supportEmail`, optional `logoUrl`, optional `footerAddress`.

17. Audit log

Single source: `platform_audit_events` table.

17.1. Actions taxonomy (`PlatformAuditAction` enum)

Action	Phase	Carried in before / after
<code>SCHOOL_STATUS_CHANGED</code>	8	<code>{ status }</code> before + after
<code>IMPERSONATION_STARTED</code>	7	<code>after.startedAt</code> , target school slug
<code>IMPERSONATION_ENDED</code>	7	<code>after.durationMs</code>
<code>SUBSCRIPTION_CREATED</code>	4	<code>after.{ plan, billingCycle, startDate, endDate, statusFlippedTo }</code>
<code>FEATURE_FLAG_CHANGED</code>	5	<code>before.overrides</code> + <code>after.overrides</code>
<code>USER_FORCE_LOGOUT</code>	9	<code>after.tokensValidAfter</code> (or <code>after.scope: "session"</code> for per-session revoke)
<code>SCHOOL_FORCE_LOGOUT</code>	9	<code>after.{ tokensValidAfter, affectedCount }</code>
<code>ADMIN_PASSWORD_RESET</code>	9	<code>after.tokensValidAfter</code> (NEVER the password)
<code>SCHOOL_MAINTENANCE_TOGGLED</code>	17	<code>before/after.{ maintenanceMode }</code>

17.2. What every audit row carries

```
{
  id, action, createdAt,
  actorUserId, actorEmail, actorRole,           // snapshotted at audit time
  targetType, targetId, targetLabel,           // label snapshotted (e.g., school name)
  before: JSON | null,
  after: JSON | null,
  reason: string | null,                       // free-form note
  ip: string | null,                           // best-effort
  userAgent: string | null,
}
```

17.3. UI

- `/platform/audit` — paginated, filterable list
- Filters: action (multi-option select), date range (from/to), free-text search across actor email + target label + reason
- Per-row `ChangeSummary` rendered per-action with custom logic
- Side-drawer with full row + before/after JSON dump

18. Migrations chronology

41 migrations, chronologically named. School-side (Phases pre-platform)

- platform layer + maturity push.

18.1. School-side foundation (Apr 22 – May 14)

Migration	Adds
20260422122834_init_multi_tenant	Schools, Users, multi-tenant base
20260422192729_teacher_optional_user	Teacher.userId optional initially
20260422195311_classes_and_sections	Class + Section
20260424094252_attendance	Attendance table
20260424102359_exams_neb_grading	Exam + ExamSubject + Result
20260425100000_exam_split_backfill	Migration to per-subject marks structure
20260425100100_exam_drop_legacy_marks	Cleanup
20260425110000_student_class_link	Student.classId + sectionId
20260425115000_fees_payments_init	FeeStructure + FeeAssignment + Payment
20260425120000_fees_class_scope_discounts	Fee class scope + discount fields
20260430000000_student_demographics	Required gender, DOB, parent, contact
20260430005000_student_symbol_number	NEB symbol number unique per school
20260430010000_school_logo	School.logoUrl
20260430020000_teacher_class_link	Legacy single-class teacher link
20260502000000_teacher_section_link	Legacy section link
20260503000000_teaching_assignments	New TeachingAssignment table
20260503144900_finalize_teaching_assignments	Schema refinement
20260504000000_teacher_user_required	Teacher.userId now required (CASCADE)
20260505000000_announcements	Announcement table
20260506000000_add_staff_role	STAFF role enum value
20260507000000_audit_fields_academic	createdById/updatedById on Subject + Exam + Result
20260508000000_academic_sessions	AcademicSession + sessionId on relevant tables
20260509000000_session_lock_promotion	isLocked + StudentAcademicRecord
20260510000000_backfill_legacy_assignments	Data migration
20260511000000_finalize_teaching_assignments_safety	Safety constraints

Migration	Adds
20260511010000_drop_teacher_legacy_class_fields	Drop legacy classId/sectionId
20260512000000_result_absent_field	Result.absent for NG handling
20260513000000_school_contact_info	School.address, .phone, .email
20260513010000_payment_refunds	Payment refund mechanism
20260513020000_payment_idempotency_and_status	clientRequestId + PaymentStatus
20260514000000_payment_audit_fields	Payment.createdById + updatedById

18.2. Platform layer (May 15 – May 20)

Migration	Adds
20260515000000_platform_layer	SUPER_ADMIN role + SchoolStatus + School.status, .expiresAt
20260516000000_platform_audit_events	PlatformAuditEvent table
20260517000000_impersonation_audit_actions	IMPERSONATION_STARTED/ENDED enum values
20260518000000_subscriptions	SubscriptionPlan + BillingCycle + SchoolSubscription + SUBSCRIPTION_CREATED
20260519000000_feature_flags	School.featureOverrides + FEATURE_FLAG_CHANGED
20260520000000_security_controls	User.tokensValidAfter + 3 audit actions

18.3. Maturity push (May 21 – May 25)

Migration	Adds
20260521000000_notifications	Notification + NotificationDelivery + 2 enums
20260522000000_jobs	Job + JobStatus enum
20260523000000_notification_severity	Notification.severity, .title, .readAt + enum
20260524000000_maintenance_mode	School.maintenanceMode + SCHOOL_MAINTENANCE_TOGGLED
20260525000000_sessions	Session table + indexes

19. Test coverage

135 tests across 12 suites. Run with `npm run test` from `backend/`.

Suite	Tests	Covers
<code>app.controller.spec.ts</code>	1	Bootstrap smoke
<code>common/money/amount-in-words.spec.ts</code>	26	Currency-to-words formatting (boundary values, lakh/crore, decimal handling, range guards)
<code>auth/auth.service.spec.ts</code>	8	Login (credential checks, tenant gate, SUPER_ADMIN bypass on suspended school)
<code>health/health.service.spec.ts</code>	13	Uptime, memory, DB probe outcomes, ring buffer windowing, status rollup
<code>feature-flags/feature-flags.service.spec.ts</code>	14	3-layer resolution, isEnabled hot path, setOverrides validation + no-op detection
<code>notifications/notification.service.spec.ts</code>	12	Template lookup, idempotency, channel fan-out, rendering, delivery state machine
<code>platform/platform.service.spec.ts</code>	4	<code>assertSchoolCanLogin</code> tenant gate (ACTIVE, TRIAL, SUSPENDED, EXPIRED)
<code>platform/security.service.spec.ts</code>	12	Force-logout (user + school), password reset, audit row safety, email best-effort
<code>platform/impersonation.service.spec.ts</code>	11	All security invariants (peer, self, nesting, suspended/expired) + happy path token shape
<code>platform/platform-analytics.service.spec.ts</code>	7	MRR computation, plan distribution, risk counts
<code>common/maintenance/maintenance-mode.guard.spec.ts</code>	13	Read pass-through, SUPER_ADMIN bypass, platform-tier bypass, write rejection
<code>sessions/session.service.spec.ts</code>	14	CRUD, throttled touch, idempotent revoke, no-leak via expectUserId, bulk revoke

⚠ Deferred:

- E2E tests (no `test/jest-e2e.json` setup configured)
- Frontend tests (no Vitest / RTL setup yet)
- Tests for older school-side modules (students, teachers, fees, attendance, exams) — shipped before the maturity push prioritized test coverage

20. Environment variables

```
# Database
DATABASE_URL=postgresql://user:pass@localhost:5432/scholaris?schema=public

# Auth
JWT_SECRET=<long-random-string>
JWT_EXPIRES_IN=7d          # default 7d

# Server
PORT=3001
APP_URL=http://localhost:3000    # used in transactional emails for absolute links

# Mail (Phase 3 maturity)
MAIL_PROVIDER=console          # 'console' (dev) or 'smtp' (prod)
MAIL_FROM='Scholaris <noreply@scholaris.example>'
MAIL_SMTP_HOST=smtp.example.com # required when MAIL_PROVIDER=smtp
MAIL_SMTP_PORT=587
MAIL_SMTP_USER=username
MAIL_SMTP_PASS=password
MAIL_SMTP_SECURE=false        # 'true' for TLS-on-connect (port 465)
MAIL_BRAND_NAME=Scholaris
MAIL_BRAND_SUPPORT=support@scholaris.example
MAIL_BRAND_LOGO_URL=https://... # optional
MAIL_BRAND_FOOTER='Acme Building, Kathmandu' # optional

# Jobs (Phase 15)
JOBS_AUTOSTART=true           # set 'false' in tests to disable poll loop

# Frontend (Next.js)
NEXT_PUBLIC_API_URL=http://localhost:3001
```

21. Operational guide

21.1. First-time setup (dev)

```
# 1. Install deps
cd backend && npm install
cd ../frontend && npm install

# 2. Configure backend env
cd ../backend
cat > .env <<EOF
DATABASE_URL="postgresql://postgres:postgres@localhost:5432/scholaris?schema=public"
JWT_SECRET="$(openssl rand -hex 32)"
PORT=3001
EOF

# 3. Run migrations
npx prisma migrate deploy

# 4. Seed the platform owner
npx ts-node prisma/seed-super-admin.ts
# → prints initial credentials

# 5. Start backend
npm run start:dev

# 6. Start frontend (in another terminal)
cd ../frontend
npm run dev
```

21.2. Restart triggers

The Nest dev server needs a restart after:

- New Prisma migration (Prisma client may need regen too)
- New module added to `AppModule`
- Changes to `JwtStrategy` / global guards
- Changes to schedule/cron registration
- Env var changes

`prisma generate` may fail with EPERM on Windows when the dev server holds the engine binary. Workflow:

1. Stop dev server
2. `rm -f node_modules/.prisma/client/query_engine-windows.dll.node.tmp*`
3. `npx prisma generate`
4. Restart server

21.3. Operator-tier daily routine

A SUPER_ADMIN logging into `/platform` typically:

1. **Glance at** `/platform/operations` — single-page health check. Status banner should be green; queue depth should be 0; failed jobs panel should be empty.
2. **Check the Notification Center bell** for unread alerts.
3. **Drilldowns** as needed:
 - `/platform/schools` for tenant management
 - `/platform/audit` for "what just happened?"
 - `/platform/health` for live error rate + login pressure

21.4. Common operations

Suspending a school:

1. Open `/platform/schools/[id]`
2. Click **Suspend** in the action bar
3. Provide a reason (required, 3+ chars)
4. School admin gets `school_suspended` email; their users can no longer log in

Reactivating:

1. Same page; status badge says "Suspended"
2. Click **Reactivate**
3. Email goes out (`school_reactivated`); users can sign in again

Resetting a school admin's password:

1. Open `/platform/schools/[id]`
2. Click **Security** → opens dialog
3. Click **Reset password** on the user row
4. Copy the temp password from the result modal (one-time)
5. Share OOB with the user (NOT email — the email has its own template)
6. The user's existing sessions are invalidated; they log in with the temp password and should change it

Investigating a tenant's data without affecting the audit trail:

1. Open `/platform/schools/[id]`
2. Click **Security** → **Sign in** (impersonation picker)
3. Pick a user; you're now seeing their dashboard
4. Banner at top reminds you you're impersonating
5. Click **Exit impersonation** when done — fresh SUPER_ADMIN token issued

Pausing writes for a tenant during a support session:

1. Open `/platform/schools/[id]`
2. Click **Maintenance** in the action bar
3. School users see a banner; their writes return 503 with a clear message

4. You can still write (SUPER_ADMIN bypasses)

5. Click **Resume** when done

22. Known limitations & deferred work

22.1. Backend deferred

- **No background queue worker tuning.** Single in-process worker. Past ~100 jobs/sec sustained, swap `JobQueueService.claimNext` for BullMQ (the abstraction is in place — see `docs/platform/jobs.md`).
- **No notification archival.** `notifications` table grows indefinitely. Future cron: move rows older than 90 days to cold storage.
- **No login anomaly detection.** Session table now records IP / UA / lastActiveAt — the data exists. The detection logic (new IP from a user who normally signs in from one location → fire `platform.security_alert`) is the next step.
- **No API token management.** No `personal_access_tokens` table. Would need scoping + token-issuance UI. Real project, not a stub.
- **No churn rate metric.** The `/platform/analytics` revenue bucket tracks current state. Computing churn requires subscription state-change history (which is captured in audit rows but not yet aggregated).
- **No successful-login trend.** We track failed logins (health buffer) but not successful ones. Adding `users.lastSeenAt` would unlock Daily Active Users + Weekly Active Users.
- **No real billing column.** `PLAN_MONTHLY_PRICE_NPR` is hard-coded in `PlatformAnalyticsService`. When real billing lands, add `school_subscriptions.priceMonthly` and the analytics service drops the constants.
- **Maintenance mode doesn't have a "scheduled" mode.** Toggle is immediate. Future: schedule maintenance for a window with auto-toggle.
- **No webhook outbox.** Producers fire fire-and-forget via the queue; no exposed webhook for downstream consumers. Phase-future.

22.2. Frontend deferred

- **No school-side notification inbox UI.** `IN_APP` channel writes to the `notifications` table; the bell badge in the topbar is currently a placeholder. The Notification Center exists for the platform side only.
- **No frontend test setup.** No Vitest / RTL configured.
- **Admin sessions list in SecurityDialog is wired backend-only.** Endpoints exist (`/platform/users/:id/sessions`) but the SecurityDialog UI doesn't yet render the list. Operator can call the API directly; UI is next.
- **No bulk session revoke for admins.** UI for "revoke all sessions for user X" — backend supports it via the watermark (force-logout) but a granular per-session list is admin-API-only.

22.3. Operational deferred

- **No CI/CD.** Manual `npm run test` + `tsc --noEmit`.

- **No E2E suite.** Critical login + impersonation flows aren't tested end-to-end against a real Postgres.
- **No production deployment guide.** Env vars are documented; the rest is left to the deployer (Docker, Kubernetes, Render, Fly, whatever).

22.4. Documentation deferred

- **No API reference doc** beyond this section. OpenAPI / Swagger generation is on the table.
- **No frontend component storybook.** The primitives are documented via inline comments + this doc; a Storybook would be the next step.

23. Future scope / roadmap

Ordered roughly by ROI. Each item is sized at "one focused engineer turn" unless noted.

High-value, ready-to-build

1. **School-side in-app inbox UI.** Render the `notifications` table for the logged-in user. Bell badge in topbar shows unread count. Backend infra is done — pure frontend work.
2. **Sessions list in SecurityDialog.** Render `/platform/users/:id/sessions` data with per-session revoke buttons. Mostly a UI extension.
3. **Login anomaly detection.** Background job scans `sessions` for "new IP from a user who normally signs in from elsewhere" → fires `platform.security_alert`. Template is ready.
4. `users.lastSeenAt` + DAU/WAU metric in analytics. Small migration
 - existing analytics service extension.
5. **Refund receipt email.** Mirror of `payment_receipt`, fired from `FeesService.refund`. Template + producer wiring.
6. **API token management.** Personal access tokens for integrations. Migration + service + UI. Multi-turn but well-scoped.
7. **Churn rate calculation.** Aggregate `SCHOOL_STATUS_CHANGED` audit rows for SUSPENDED transitions in the last N days. Pure analytics work.

Medium-value, more design work needed

8. **Real billing integration.** `priceMonthly` column on subscriptions. Consider Stripe / eSewa integration + invoice generation.
9. **Webhook outbox.** Let downstream systems subscribe to platform events.
10. **Scheduled maintenance windows.** Toggle on/off auto-fires based on `start` and `end` timestamps.
11. **Multi-admin notifications.** Currently emails go to "first ADMIN" — multi-admin schools should fan out.
12. **CSV bulk import** for students / teachers (school-side). UI + validation + transactional import.

Infrastructure

13. **CI/CD pipeline** (GitHub Actions): typecheck + test on PR.
14. **E2E test setup** with a real Postgres.
15. **Frontend test setup** (Vitest + RTL).
16. **OpenAPI / Swagger** generation for the API.
17. **Storybook** for the design primitives.
18. **Docker Compose** for the dev environment (Postgres + backend + frontend in one `docker-compose up`).
19. **Background queue** → **BullMQ migration** when load demands it.
20. **Notification archival cron**.

Product expansion

21. **SMS channel** — wire Twilio / a Nepal-local SMS provider (Aakash, Nikatel). Template scaffold is in place.
22. **WhatsApp Business** channel for parents.
23. **Transport / hostel modules** — they're already in the feature catalog as "coming soon."
24. **Mobile apps** (React Native / Flutter) reusing the existing API.
25. **Public school directory** + signup flow for self-serve onboarding.

24. Decision log

Key architectural choices and the reasoning behind them. Documented so future contributors don't accidentally undo decisions made deliberately.

Why two completely separate UI surfaces (school + platform)?

Different audiences (school admins vs platform owners), different data scopes (single tenant vs cross-tenant), different visual languages (welcoming vs operational), different security postures. Sharing components would invite accidental privilege escalation (e.g., a component that doesn't filter by `schoolId` because the platform side doesn't need to).

Why `SUPER_ADMIN` as a Role enum value (not a separate table)?

Symmetric with other roles. A `SUPER_ADMIN` still has a User row, an email, can authenticate the same way. Putting them in a separate table would duplicate the auth machinery. The trade-off: `SUPER_ADMIN`s are technically "at" some school (FK requirement), but that `schoolId` is irrelevant to platform routes. Documented in the Role enum.

Why append-only subscriptions (no edit endpoint)?

Audit trail naturally reflects lifecycle without a versioning column. A future "what plan did this school have on date X?" query is a range query. A future PDF receipt generator can reconstruct any historical period. The cost: the UI has to be careful that "edit" actually means "create a new period that supersedes."

Why DB-backed background queue (not Redis)?

Two reasons: (1) Postgres is already in the stack — no new infra at v1's volume; (2) the `jobs` table is operator-visible (`SELECT * FROM jobs WHERE status='FAILED'` beats grepping logs). When we hit ~100 jobs/sec sustained, swap `JobQueueService.claimNext` for BullMQ — the `JobHandler` interface stays the same.

Why hard-coded plan prices in code (not DB)?

V1 doesn't have real billing. Hard-coding `PLAN_MONTHLY_PRICE_NPR` makes MRR computation work today without a column-change migration. When real billing lands, add `priceMonthly` to `school_subscriptions` and remove the constants.

Why `tokensValidAfter` watermark instead of revocation table?

Two needs:

- "Kill ALL of a user's tokens" → watermark is constant-cost (one column, no growth). Phase 9.
- "Kill THAT one token" → sessions table (Phase 17).

Both coexist; the strategy honors both. A pure revocation list would grow unbounded without periodic cleanup.

Why `SessionService.touch` throttles to 1/min?

Without it, EVERY authenticated request would write to the `sessions` table. Write amplification on what should be a cheap read path. The throttle caps at one update per minute per session — operator-visible "last active" stays current enough to be useful (within ~1 min).

Why JWT-only (no refresh tokens)?

Simpler. The session table already gives us per-session revoke + admin control. Refresh tokens add complexity (rotation, secure storage, re-issuance) for marginal benefit at this scale. Could revisit if tokens need to be shorter-lived than 7d.

Why no separate audit table per phase?

A single `platform_audit_events` table with a polymorphic `(targetType, targetId)` + JSON `before/after` was a deliberate choice. Adding a new audit-able action is one new enum value + one `PlatformAuditService.record()` call site — never a new table. The trade-off: the table mixes heterogeneous shapes; queries that want a specific action's data have to JSON-extract.

Why feature flags resolved live (not cached)?

Resolution is one indexed lookup. Caching would add invalidation complexity (when an override changes, which servers' caches need flushing?). A future Phase that adds Redis + horizontal scaling can add a TTL-based cache; until then, fresh reads are correct.

Why is `FeaturesProvider` mounted in the dashboard layout (not `_app`)?

The login page doesn't need feature flags — those routes live outside the `(dashboard)` route group. Mounting at the dashboard layout level keeps the fetch tied to authenticated traffic only.

Why does impersonation NOT carry a special audit role?

Domain audit columns (`createdById` on payments etc.) get the TARGET user. Reasoning: impersonation is for "reproduce what the school admin sees + does"; backdating writes to the SUPER_ADMIN would make the school's own audit trail incoherent. The platform-side `IMPERSONATION_STARTED` / `IMPERSONATION_ENDED` rows bracket every write, so the combined story is complete.

Why is the global `MaintenanceModeGuard` a guard (not middleware)?

Guards run AFTER `JwtAuthGuard` so `req.user` is populated. A middleware would have to either re-decode the JWT or duplicate the tenant lookup.

Why custom `UserAwareThrottlerGuard` ?

Default `ThrottlerGuard` keys by IP. In production a school behind one NAT shares ONE bucket across every teacher's laptop — one busy user starves others. Custom subclass keys by `req.user.id` for authenticated requests. Auth/register stays IP-keyed (no identity yet).

Appendix A — How to use this document

This doc is meant to be **navigated**, not read top-to-bottom. The table of contents is the entry point.

- **Onboarding a new engineer:** §3 (principles), §6 (auth), §10 (modules), §21 (ops).
- **Adding a feature flag:** §15. Read the catalog file, add an entry, decorate a controller with `@RequireFeature(...)` , hide the sidebar entry on the frontend.
- **Adding a new email template:** §13.4. Drop a template file, register, call `notifications.enqueue` from your producer.
- **Adding an audit-able action:** §17.1. Add an enum value to `PlatformAuditAction` , generate a migration with `ADD VALUE` , emit via `PlatformAuditService.record({action: 'YOUR_NEW_ACTION', ...})` .
- **Investigating a bug:** §22 first (might be a known limitation), then `docs/platform/architecture.md` for "where to look when X breaks."
- **Planning the next phase:** §23.

Keep this doc updated. Every new phase / migration / model / route should add an entry here. The cost of stale documentation is much higher than the cost of keeping it current.

Appendix B — File metrics snapshot

As of last update:

- **41 migrations** (11 school-side + 6 platform layer + 5 maturity = 22 since pre-platform; the rest predate)
 - **12 test suites, 135 tests** all passing
 - **Backend modules:** ~25 feature/cross-cutting modules
 - **Frontend pages:** ~40 distinct routes across two surfaces
 - **Email templates:** 9 production-ready
 - **Background job handlers:** 2 (`notification.send_delivery` , `platform.subscription_expiring_notice`)
 - **Documentation files:** 5 (this + 4 in `docs/platform/`)
-

Last updated: this doc tracks the platform state as of the last "continue" iteration. When you ship the next phase, append to §9 (or add §10) and update §22/§23.